# 洛神 O1

## 会死的数字生命引擎

稀缺 · 死亡 · 不可逆 · 代谢 · 内生约束

2026 开源行业解决方案创新赛 · 赛道一
开源基础软件与解决方案

| 项目 | 信息 |
| --- | --- |
| 项目负责人 | 朱美岩（独立开发者） |
| 开源协议 | Apache-2.0 |
| 代码依赖 | 仅 numpy —— 零 LLM 依赖、零网络依赖 |
| 失败模式库 | 226 条，全部带代码可复现 |
| 实验报告 | 64 份，含被推翻的预注册判据 |

**“所有开源 Agent 框架都在问「它能做什么」。**
**本项目问的是「当它必须取舍、必须记住、而且真的会死的时候，它是什么」。**

# 60 秒速览

一句话：一个只有 numpy 依赖的单文件引擎，用「稀缺 · 死亡 · 不可逆 · 代谢 · 内生约束」五条硬约束，让数字系统被迫长出结构——关掉全部决策模块，取舍依然出现。

| 最硬的三件事 | 实测数字 | 怎么验证 |
| --- | --- | --- |
| 它真的会死 | 16 个种子跑 600 拍，3 个真死（seed 3 死于第 525 拍）；死亡有诊断书、退出码 0、引擎继续 | python3 experiments/run_core.py |
| 不缺就不留疤 | 撤掉稀缺后结构偏度精确归零 0.0000；撤掉任一条，四条配置全部 0.0000 | 同一脚本，实验 C |
| 结构有关键期 | 早期引导撤除（1.0933）强于全程施加（0.8502）；起点 400 拍后门已关上 | python3 examples/bio_hypothesis_test.py |

怎么验证（3 秒，不要求信任本文档任何一句话）：**python3 tools/verify_all.py** ——在你自己的机器上当场跑出上面每一个数字，并对四条核心结论做断言，共 11 项。我们做过负对照：把结论③的疤痕强制归零后重跑，该项立即变红、退出码 1，不会恒绿报喜。

它挂在哪：不到 20 行，不改原框架一行代码，把任意 Agent 的「我要多少」换成「我分到了多少」——examples/governed_agent.py 可直接跑。

它给谁用：首选场景是生物科学假说检验——湿实验贵、慢、混杂变量多，这里把机制单独剥离，变量完全可控、逐位可复现。三条命题（关键期 / 剂量-反应 / 选择压开关）已实测，见第 5.1 节。

它与其它参赛作品最大的不同：我们开源了全部失败——226 条带代码的失败模式、64 份含被推翻预注册判据的实验报告。在 AI 生成内容泛滥的 2026 年，三十秒能编一个成功故事，但编不出 226 条失败记录。

# 一、项目背景：我们要解决的不是"更聪明的 Agent"

当前所有开源智能体框架（LangChain、AutoGPT、Dify
等）解决的是同一个问题：如何让大模型更好地调用工具、规划任务、记住上下文。

它们的共同前提是：这个 Agent 不会死。

| 其它框架 | 本引擎 |
| --- | --- |
| 上下文超了 → 截断 / 压缩 | 资源耗尽 → 运行终止，不可回滚 |
| token 用完 → 换 key 继续 | 能量必须由自身产出换取，无外部拨款 |
| 状态丢了 → 重新加载还是"同一个" | 结构容量只减不增，丢掉的回不来 |

一个不会死的系统，不需要做取舍。而不做取舍的系统，不会产生偏好，也不会形成结构。

本项目要解决的正是这个被整个行业跳过的问题：当资源有限、时间不可逆、死亡是真实结局时，一个数字系统会如何自发地长出结构？ 这不是在现有 Agent 框架上加一个插件，这是换了前提。

# 二、技术架构：五条硬约束

五个彼此独立、缺一不可的条件，全部是硬约束而非惩罚项：

| 条件 | 实现 | 性质 |
| --- | --- | --- |
| 稀缺 | 每拍预算 B = ratio × Σwant，通道间零和的硬取舍<br>（分不到的 = 0，而不是少给一点） | 硬约束 |
| 死亡 | E ≤ 0 持续超过耐心阈值 → 运行终止，不回滚 | 真实结局 |
| 不可逆 | 长期饥饿 → 结构容量 capacity 永久缩小，只减不增 | 单调 |
| 代谢 | E 只能由系统自身产出换取，无外部拨款 | 闭环 |
| 内生约束 | 设定点 sp ← ref ← y，y 有 leak（不持续投入就衰减），外部不可写入 | 自生 |

实现形态：单文件核心引擎 core/life5.py，唯一依赖 numpy。无 LLM、无网络、无外部服务 —— 任何机器 pip install numpy 后即可复现全部结论。

关于「可部署」，我们验证过而不是写过：`pip install` 的退出码为 0 不等于装出来的包能用。本仓库曾长期缺 `core/__init__.py`，`find_packages()` 返回空列表，`pip install .` 装出一个不含任何代码的空包 —— 而当时文档仍在声称「可 pip 安装」。现已修复，并由 `tools/verify_all.py` 第 ④ 组守护：真装一次到临时目录，断言 `from core import Life5` 可用；删除 `__init__.py` 立即 FAIL。这条失败记录保留在库里（判据核第 278–281 条），不作删除。

# 三、核心创新点

## 创新一：结构不是被设计的，是被约束逼出来的

我们做了一个反直觉的实验：把全部"决策模块"关掉。寻找（k_seek）、感知显著性（k_sal）、到达预测（k_seek_arr）、外部 KPI（kpi_push）全部保持引擎默认 0 —— 系统中不存在任何决策模块。前 300 拍分别饿通道 3 / 通道 5，之后两组的运行条件完全相同，测同一个通道 3 的后段份额：

| 前 300 拍 | 通道 3 后段份额 |
|---|---|
| 饿通道 3 | 0.00004 |
| 饿通道 5 | 0.1495 |
| Δ | -0.1494 ± 0.0311（SE, n=8） |

8 / 8 seeds 符号一致。饿过谁，之后就少给谁。而这段"记忆"不在任何模块里 —— 它在不可逆结构容量里。没有决策模块、没有规则参数、没有强化学习。取舍是稀缺 × 不可逆 × 竞争动力学的投影。

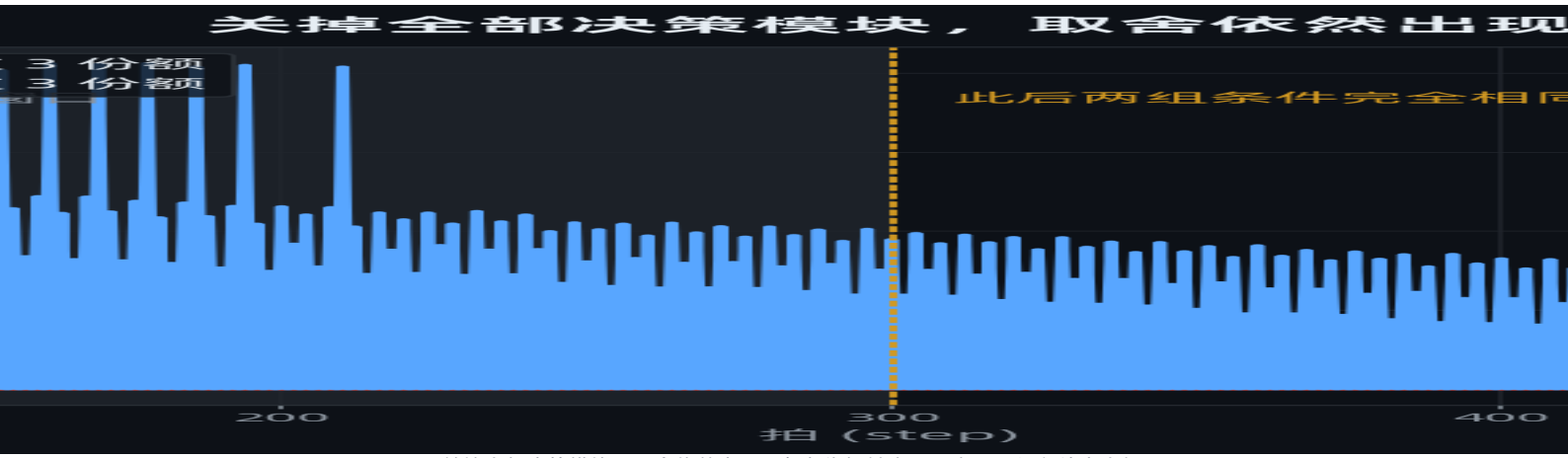



图 1 · 关掉全部决策模块，取舍依然出现（灰色为饥饿窗口，之后两组条件完全相同）

## 创新二：稀缺与不可逆是焊死的，不是两个开关

| 配置 | 终态结构偏度（std/mean of capacity，n=8） |
|---|---|
| 稀缺 在 不可逆 在 | 0.2913 ± 0.0284 |
| 稀缺 在 不可逆 撤 | 0.0000 |
| 稀缺 撤 不可逆 在 | 0.0000 |
| 稀缺 撤 不可逆 撤 | 0.0000 |

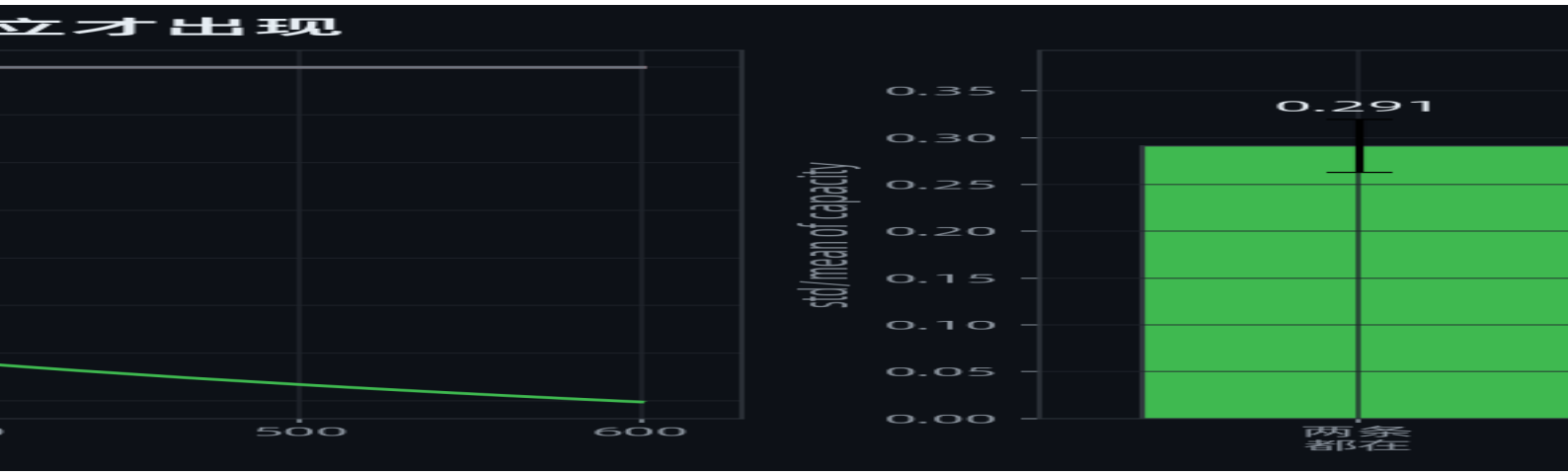



图 2 · 结构疤痕：只有两条约束同时成立才出现

撤掉稀缺 → 永远不饿 → 疤痕从不形成；撤掉不可逆 → 结构在撤除瞬间塌回均匀。"不缺，就不留疤。"两条互为存在条件 —— 这也是"不可逆"必须被实装在结构上（而不是装在饿感上）的原因。

左图：四条配置下 capacity 的逐拍轨迹 —— 只有深绿那条（两条约束都在）一路下行，其余三条全程为一条水平线，即疤痕从未形成。右图：终态结构偏度，同样的结论以数值呈现。

## 创新三：结构有关键期，早期施加后撤除的效果强于全程施加

| 引导窗口 | 终态结构偏度（n=8） |
|---|---|
| 早期 0–150，之后撤除 | 1.0933 ± 0.0105 |
| 中期 150–300 | 0.5092 ± 0.0468 |
| 晚期 300–450 | 0.3243 ± 0.0500 |
| 全程施加 | 0.8502 ± 0.0070 |
| 全程无引导（基线） | 0.2913 ± 0.0284 |

早期给过、撤掉之后结构留下了（0.2913 → 1.0933，+0.802）；晚期给几乎无效（0.3243 ≈ 基线 0.2913）—— 过了关键期，门就关上了；而早期施加后撤除（1.0933）反而强于全程施加（0.8502），因为持续投入会摊薄集中度。

机制即不可逆：容量只减不增，一度形成的形状被锁死。这对"如何养育一个系统"给出了可操作结论：引导要早，且要敢于撤。

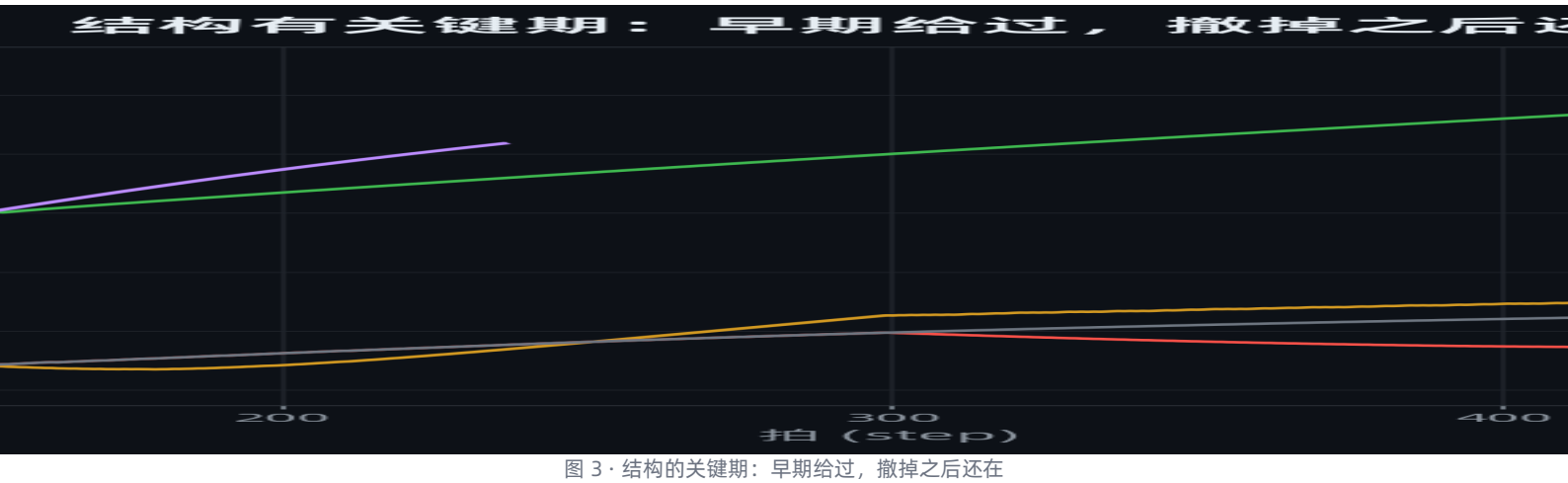



图 3 · 结构的关键期：早期给过，撤掉之后还在

## 创新四：死亡是真实结局，不是异常分支

16 个种子真实跑 600 拍：13 活 / 3 死（seed 3 死于第 525 拍、seed 4 死于 596、seed 7 死于 516）。系统会死，这是特性不是缺陷 —— 因为只有会死的系统才需要取舍。

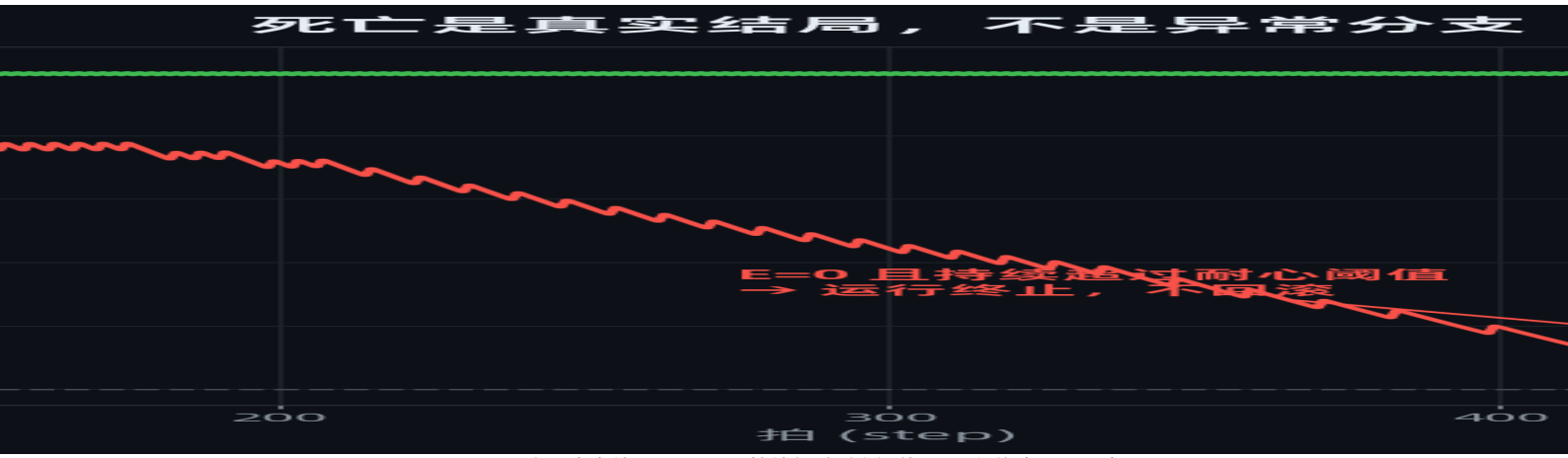



图 4 · 死亡是真实结局：E ≤ 0 且持续超过耐心阈值 → 运行终止，不回滚

## 创新五：外挂越可靠，撤走后越危险 —— 而撤走前看不出来

外部接口（工具、API、凭据）会替系统承担一部分工作。我们把这件事做成了可测的机制：宿主承担摄食通道的 off 比例工作，200 拍后统一撤走。结果两条曲线方向完全相反。

| 外挂承担比例 off | 撤走后存活 | 撤走【前】能量 | 其他通道结构 |
|---|---|---|---|
| 0（无外挂） | 0.812 | 0.8927 | 0.2943 |
| 0.15 | 0.500 | 0.9628 | 0.3297 |
| 0.25 | 0.375 | 0.9793 | 0.3329 |
| 0.30 | 0.250 | 0.9832 | 0.3411 |
| 0.50（重度依赖） | 0.000 | 0.9893 | 0.3685 |

外挂越重，撤走前能量越高（0.8927 → 0.9893）、其他通道结构越好（0.2943 → 0.3685，这是外挂的真实收益），而撤走后存活率越低（0.812 → 0.000）。

| 用撤走前的量预测"撤走后能否活下来" | 相关系数 | AUC |
|---|---|---|
| 撤走前表观能量 | -0.014 | 0.308（低于随机） |
| 其他通道结构（外挂的收益） | -0.877 | 0.023（近乎完美反向） |
| 被卸载通道自身的结构 | +0.719 | 0.939 |

最终存活者与最终死亡者，撤走前的平均能量分别是 0.9604 与 0.9616——差 0.0012，噪声内且方向相反。只看产出，完全区分不出来。

对照组（外挂全程不撤走）在 off≥0.10 时存活率恒为 1.000。所以这不是"外挂有害"，而是：外挂的风险全部在撤走那一刻兑现，不在使用期间。

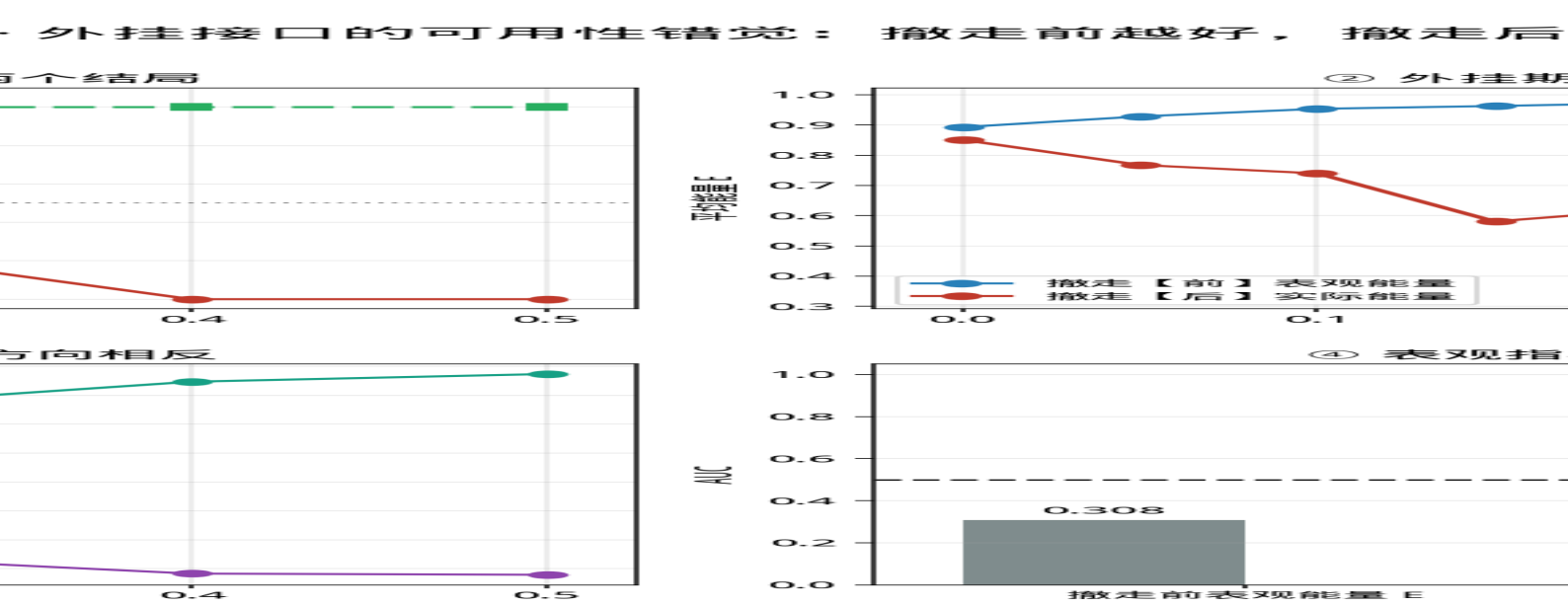



图 5 · 外挂的可用性错觉：撤走前越好，撤走后越惨

# 创新六：与同类平台的定位差异 —— 不是更强，是测的东西不一样

赛道一里与本引擎最相近的其实不是 LangChain / Dify，而是人工生命（ALife）数字演化平台：Avida（Ofria, Bryson & Wilke, 2009）、Tierra（Ray, 1991）、Evochora、Lenia。评委一定会问："Avida 不就能做这个吗？"

我们不主张"我们更好"。我们主张一件更弱、但可证伪的事：在 Avida 式设定下，我们要测的那个东西根本不出现。

## 设定差异（据公开资料）

| | Avida 式 | 洛神 O1 |
|---|---|---|
| 能量 | 外生奖励：完成任务 → 更多 CPU 周期 → 复制更快 | 代谢预算：E 只能由自身产出换取，且有 basal 基础消耗 |
| 结局 | 被替换 / 淘汰（个体不会饿死） | 死亡：E ≤ 0 超耐心阈值 → 永久终止，不可回滚 |
| 损耗 | 可逆（变异可回退、换代重置） | 不可逆：capacity 只减不增 |

## 2×2 实测（16 seeds × 600 拍，约 2 秒）

| 设定格 | 死亡 | 永久损失 | 容量回升 |
|---|---|---|---|
| 内生代谢 × 不可逆（本引擎） | 3/16 | 0.7043 | 恒为 0 |
| 内生代谢 × 可逆 | 4/16 | 0.5472 | 0.0048 |
| 外生奖励 × 不可逆 | 0/16 | 0.6990 | 恒为 0 |
| 外生奖励 × 可逆（≈ Avida） | 0/16 | 0.5453 | 0.0046 |

能量来源决定"会不会死"，可逆性决定"伤痕是否永久"。在 Avida 式设定下，代谢性死亡 0/16 —— 不是数值更低，是结构性不发生。

定位声明：本引擎不是 Avida 的替代品，而是 Avida 设定下不存在的那一层。所谓技术优势，不是"跑得更快、更复杂"，而是我们测的现象在它们的设定下没有定义。

诚实标注：这是我们对 Avida 式设定的建模，尚未与 Avida 本体对接实测 —— 已列为最欢迎的外部贡献（见 CONTRIBUTING.md）与 ROADMAP v0.6 首要缺口。本轮还推翻了我们自己的一条预注册判据：初版用离散度 std/mean 量"疤痕"，实测发现可逆模式反而拉大离散度（0.4990 > 0.2695），离散度不是不可逆性的度量，已改用"是否出现回升"与"永久损失"。

第二条诚实标注（覆盖范围的边界）：上述声明只覆盖 Avida / Tierra，不覆盖 Evochora。据 Evochora 官网，其"mutation mechanics, resource dynamics, and death conditions are fully programmable through swappable plugins"——即在 Evochora 上可经插件配置实现代谢性死亡。故本引擎相对 Evochora 的差异不是"能不能死"，而是"死亡是否由内生代谢预算强制、损耗是否不可逆地刻在结构容量上"；这一点我们尚未在 Evochora 上实测，不作主张。Lenia（Bert Chan, 2018）为连续细胞自动机，不以"个体-资源-死亡"为建模单元，不在同一问题域。

文献出处（核实于 2026-10-16）：Ofria, Bryson & Wilke (2009), Avida: A Software Platform for Research in Computational Evolutionary Biology, in Artificial Life Models in Software (Springer), pp. 3-35, DOI 10.1007/978-1-84882-285-6_1。

# 创新七：取舍驱动结构，不是稀缺驱动结构

前六个创新点都建立在一条表述上："稀缺驱动结构，富足杀死结构"。本轮我们把这句话拆开检验，发现它是混淆了两个可分离的变量，并据此修正。

## 可证伪的二分

"富足杀死结构"里的致死因子，到底是资源多，还是不取舍？若前者 —— 富足下即便强制取舍，结构仍应归零；若后者 —— 富足下只要强制取舍，结构就该保住。我们用引擎的 alloc_fn 把"取舍"变成可独立于资源量操作的变量：Dense（等比缩放，永不取舍）对比 TopK（只激活 want 最大的 K 个通道，强制取舍）。

## 实测：ratio 跨 4 倍，两条曲线各自水平（16 seeds × 600 拍）

| ratio（资源充裕度） | Dense · 无取舍 | TopK(K=3) · 强制取舍 |
|---|---|---|
| 0.30（最紧） | 0.0005 ± 0.0001 | 0.2459 ± 0.0150 |
| 0.45 | 0.0005 ± 0.0001 | 0.2836 ± 0.0153 |
| 0.60 | 0.0005 ± 0.0001 | 0.2831 ± 0.0169 |
| 1.20（富足） | 0.0000 ± 0.0000 | 0.2836 ± 0.0168 |

资源量几乎不起作用：ratio 从 0.30 走到 1.20，两条曲线各自水平、互不相交。真正决定结构的是是否发生取舍。因：等比缩放是尺度变换，而 std/mean 尺度无关，故 Dense 在任何预算下都不改变相对结构；初始 capacity 偏度为 0，故 Dense 的情形是"结构从未生成"，不是"被杀死"。

修正后表述：取舍驱动结构；资源量只在"是否迫使取舍"的意义上间接相关。第 285 条据此修正第 044 条相关表述。

## 来自顶级开源模型的独立印证

2026 年顶级开源模型的默认底盘是 MoE 稀疏激活：DeepSeek-V4 Pro 1.6T/49B、Kimi K3 2.8T/104B、Qwen3.5 397B/17B，激活比仅约 3%–5.5%。关键事实是：即便算力充足，它们仍然强制 top-K 路由。

| | 顶级开源模型 | 洛神 O1 引擎 |
|---|---|---|
| 强制取舍 | MoE top-K 路由 | TopK 分配（K<N） |
| 不取舍 | Dense 全激活 | Dense 等比缩放 |
| 取舍的结果 | 专家分化、效率更优 | 结构偏度 0.24–0.29 |
| 不取舍的结果 | 无专家分化 | 偏度 0.0000 |

MoE 之所以在算力充足时代仍然胜出，不是因为省算力，而是因为它保住了"取舍"。一个规模大若干个数量级、经数千工程师验证过的独立系统，收敛到了与本项目引擎同一条机制。

我们取的是它已验证的规律，不是它的实现（第 287 条）：抄实现是复制，取规律并以自己的引擎独立检验才是捷径。

诚实标注：本实验推翻了我们自己的一条预注册预测 —— 原预测"稀缺下 Dense 偏度也应 > 0"，实测为 0.0005。该推翻反而是关键证据：它证明"稀缺"与"取舍"可分离，使本条从"相关"升级为"因果"。另有一处未解释异常：富足下剂量反应 K=8→2 中 K=4 出现局部凹陷（0.0000 / 0.0839 / 0.1097 / 0.1272 / 0.0485 / 0.1870 / 0.1816），已排除死亡筛选（t=300 复测异常保留）与存活率差异，未找到解释，不作解释性主张。顶级开源模型的参数量引自公开选型报道，未逐一核对官方技术报告原文，可能有出入。

# 四、我们开源了全部失败 —— 226 条带代码的失败模式

evidence/失败模式库/判据核.md 收录 226 条失败模式，每一条都有代码可复现。配套 64 份实验报告，每一份都记录了被推翻的预注册判据。

| 失败模式 | 症状 | 我们栽过 |
|---|---|---|
| 饱和伪装成"无效应" | 信号 ×1 / ×10 / ×100，指标一字不差 | ≥ 3 次 |
| 变量静默失效 | 变量写了但没接进主通道，不报错、不崩、图照出 | ≥ 4 次 |
| 相位假象 | 测量窗口恰好落在振荡的某一半，结论随取哪一代反转 | ≥ 3 次 |

在 AI 生成内容泛滥的 2026 年，任何人三十秒就能编出一个可信的成功故事。但编不出 226 条带代码的失败记录。 我们认为这是开源项目最稀缺的资产类型。

我们还公开了对第三方方法论的审计，并已取得对方真身做实测（不是模拟）。对象是赛道内同类公开作品 TACT 三轴坐标分类法（CC BY 4.0，作者 KLameng，致谢）。我们经公开接口取得其坐标库（259,226 字节）与引擎源码，复现其公开基线 L 78.6% / M 95.7% / P 86.4% 逐位一致 —— 确认取到的确是真身后，做了三项检验：

| 检验 | 结果 |
|---|---|
| 分轴召回 vs 三轴联合 | 分轴 78.6 / 95.7 / 86.4，但坐标是三元组，三轴完全匹配仅 14/29 = 48.28%；三轴独立时的联合期望是 64.91%，实际低 16.6 个百分点（轴间错误正相关） |
| 同义改写（语义不变，只换词面） | 三轴召回全部精确归零；剔除 golden 全空的 2 条后 44.44% → 0.00% |
| 标点扰动（无关对照，不改任何词） | 与原文逐位相同 —— 证明效应来自词面替换，而不是来自"改写"这个动作 |

剂量-反应：累积替换 k 个语义等价的词，翻转率 k=1 → 16.7%、k=3 → 50.0%、k=6 → 100%（12/12 全部翻转）。每条都存在有限的翻转阈值（1~6 个词）。结论：它命中的是词，不是语义。

必须说明：这一点对方已在 SELF_FAILURES.md 的 SF-01 中自我承认（"k 是评审者彼此一致，不是跟真相一致，内部一致性 ≠ 正确性"）。我们与对方独立收敛到同一结论，故本轮定位为独立确认与方法贡献，不是揭发；新增价值只在对方未写的两处：联合匹配率被分轴指标掩盖、字面匹配的改写敏感性。

同一把尺子量自己：本实验的前提"改写是否语义等价"，是由我们（单一模型、无第三方人工验证）自行判定的 ——
这正是我们自己在第 269 条记录的自证病。29
条改写文本与全部结果均已公开，任何人可复核；但根除它需要独立标注者，我们未做到。

关于条数，如实说明（这一条本身就是我们主张的实践）：226 是独立条目数，编号范围 35-287；其中 27 处编号缺口（52、116-117、147-163）、10
个重复编号，我们不回填、不重排——回填即改写历史，违反"只增不减"。第 1-34 条的外部引用链已断裂，不计入本数。清点可复现：python3 tools/count_patterns.py（权威脚本；旧脚本 count_criteria.py 因正则漏掉行首加粗写法而少计，仅作交叉校验保留）。我们此前对外称"195 条"，是把最大编号误当条目数；随后清点脚本又因正则漏掉行首加粗写法而少计 2 条。两处均已修正，现以 tools/count_patterns.py 输出为准。演示视频片头标注 159 条，是录制当时的数值；库随实验持续增补，截至本稿为 226 条。两者差异已如实披露，不做静默对齐。视频内的全部实验数据均为真实运行输出，与条数无关。

# 五、应用场景与落地

| 场景 | 本引擎提供的能力 |
| --- | --- |
| 生物科学假说检验（首选） | 给演化 / 发育 / 生态命题提供变量完全可控、逐位可复现、成本近乎为零的 in silico 对照沙盒——见 5.1，三条命题已实测 |
| 长期自治 Agent 的资源治理 | 根治"上下文无限膨胀、无取舍、无偏好"—— 资源预算是硬约束，不是软提示 |
| 不可逆审计与合规 | 结构容量只减不增 + 时间线单调，天然提供不可篡改的行为记录 |
| 多智能体多样性研究 | 同一组硬约束下由初始差异自发产生个体分化，可用于群体鲁棒性设计 |
| AI 存在形式化研究 | 五条件可作为"数字系统是否具备生命特征"的一组可测判据 |
| 教学与科普 | 零依赖、可交互，适合作为"人工生命 / 复杂系统"课程的可运行教具 |

可运行性验证：python3 tests/test_engine.py —— 引擎自检 11 项全部通过，逐条证明五条硬约束确实成立，而不是文档里写着成立。

## 5.1 生物科学：三条命题的 in silico 对照检验

湿实验贵、慢、混杂变量多。本引擎的价值不是替生物学家下结论，而是把机制单独剥离出来，让假说可以被反复、逐位复现地检验。以下三条全部由 examples/bio_hypothesis_test.py 当场跑出，数据读自 data/bio_results.json，未写死任何一个数。

H1 · 发育关键期是否存在、门何时关上

生物学对应：视觉剥夺实验、印记（imprinting）的关键期。固定引导窗口 150 拍，只扫起点，测终态结构偏度（n=8）：

| 引导起点 | 结构偏度 | 相对基线 |
| --- | --- | --- |
| 0 | 1.0933 | +0.8020 |
| 100 | 0.6311 | +0.3398 |
| 200 | 0.4073 | +0.1160 |
| 300 | 0.3243 | +0.0330 |
| 400 | 0.2892 | -0.0021 |
| 450 | 0.2698 | -0.0215 |
| 无引导基线 | 0.2913 | – |

效应单调衰减，起点 400 拍之后衰减到基线 +1SE 以内（即门已关上）：起点 0 时 +0.8020，起点 300 时仅 +0.0330。这给出一条可检验的预测——存在关键期边界，且边界可被定位。

H2 · 剂量-反应：早期剥夺是线性还是阈值

生物学对应：代谢编程 / DOHaD（早期营养剥夺的长期效应）。饿探针通道不同时长，测其后段份额损失（饿自己 – 饿别人，n=8）：

| 饥饿拍数 | Δ份额 | SE | 符号一致 |
| --- | --- | --- | --- |
| 50 | -0.1342 | 0.0232 | 8/8 |

| 饥饿拍数 | Δ份额 | SE | 符号一致 |
|---|---|---|---|
| 100 | -0.1454 | 0.0249 | 8/8 |
| 200 | -0.1485 | 0.0284 | 8/8 |
| 300 | -0.1494 | 0.0311 | 8/8 |
| 400 | -0.1495 | 0.0333 | 8/8 |

8/8 seeds 符号一致。但它是阈值型而非线性：最短剂量 50 拍已达最大效应的 89.8%，延长到 400 拍只再增加 0.0153。即早期剥夺存在阈值，超过后效应饱和——与 H1 的关键期互为佐证。

H3 · 选择压是旋钮还是开关

生物学对应：选择压强度与生态位分化。扫描稀缺度 ratio（n=8）：

| ratio | 存活 | 结构偏度 | SE | 死亡/8 |
|---|---|---|---|---|
| 0.30 | 6 | 0.2383 | 0.0119 | 2 |
| 0.45 | 5 | 0.3069 | 0.0445 | 3 |
| 0.60 | 7 | 0.1520 | 0.0479 | 1 |
| 0.80 | 8 | 0.1854 | 0.0429 | 0 |
| 1.00 | 8 | 0.0000 | 0.0000 | 0 |
| 1.20 | 8 | 0.0000 | 0.0000 | 0 |
| 2.00 | 8 | 0.0000 | 0.0000 | 0 |

结论与直觉相反：稀缺是开关，不是旋钮。ratio < 1.0 时结构一概成形，但强度不随稀缺程度单调变化；ratio ≥ 1.0 时结构精确塌回均匀（0.0000），且无一死亡。这正是本项目主张「稀缺是硬约束而非惩罚项」的直接证据——惩罚项才有剂量-反应，硬约束只有成立与不成立。

定位声明：以上不构成生物学成果，而是为生物学假说提供一处可复现的对照沙盒。本引擎不含任何真实生物参数，与生物系统仅为结构类比。

## 关键：它不是又一个框架，是可以挂到任何框架上的一层

本引擎的真正形态不是"替代 LangChain / AutoGPT"，而是给任意 Agent 框架补上它缺的那一层。核心不到 20 行，且不需要改动原框架的一行代码：

```
import numpy as np
from core.life5 import Life5

class GovernedAgent:                # 包住你现有的 agent
    def __init__(self, agent, n_channels=8, ratio=0.45, seed=0):
        self.agent = agent          # LangChain / AutoGPT / 自研 agent
        self.life  = Life5(N=n_channels, ratio=ratio, seed=seed)

    def step(self, obs):
        wants = self.agent.propose(obs)       # 原框架照常说"我要多少"
        self.life.input_val = float(np.mean(wants))
        rec = self.life.step()        # 稀缺→硬取舍→不可逆→代谢→死亡
        if rec is None or not self.life.alive:
            raise RuntimeError(f"agent terminated at t={self.life.death_t}")
        return self.agent.act(obs, rec["avec"])  # 只拿分到的份额去行动
```

改动只有一处：原框架说"我要多少"，现在改说"我分到了多少"。该示例已实装为 examples/governed_agent.py，可直接运行。实测输出：

| seed | 结局 | 终止拍 | 终态结构偏度 |
|---|---|---|---|
| 3 | 死亡 | 525 | 0.2723 |
| 5 | 存活 | – | 0.2792 |
| 11 | 存活 | – | 0.2528 |
| 13 | 存活 | – | 0.2322 |
| seed 3 · 撤掉稀缺（ratio=1.2） | 存活 | – | 0.0000 |

最后一行是本项目最重要的一条结论的现场复现：不稀缺就没有取舍，没有取舍就不留疤 —— 撤掉稀缺后，结构偏度精确归零，系统不再成形。

这也是为什么本项目属于「基础软件」而非「应用」：它改的是 Agent 的运行时前提，不是给它加一个功能模块。

## 六、开源治理

| 项目 | 状态 |
|---|---|
| LICENSE（Apache-2.0） | 已完备 |
| CONTRIBUTING.md 贡献指南 | 已完备 |
| README 与依赖清单 | 已完备（依赖仅 numpy） |
| 可运行测试 | tests/test_engine.py，11 项自检 |
| 失败模式库开源 | 226 条，带代码可复现 |
| 实验报告开源 | 64 份，含被推翻的预注册判据 |
| 复现清单 | docs/复现清单.md，逐条标注每个数字怎么来的 |
| 评委即开即用演示台 | docs/demo.html，单文件零依赖，双击即开 |

## 七、长期发展

| 阶段 | 目标 |
|---|---|
| 近期（3 个月） | 引擎打包上传 PyPI；接入真实世界数据源作为稀缺输入；补齐外部平台基线（Avida / ABM）对比 |
| 中期（6-12 个月） | 发布完整失败日志的复现代码集；把"五条件"做成可插拔中间件，供任意 Agent 框架接入 |
| 长期 | 推进"数字生命状态方程"的学术发表与标准化尝试 |

判据核本身采用"只增不减、改动需本人 merge"的不可变核治理方式 —— 这与本项目主张的"不可逆"是同一条原则在治理层面的应用。

## 八、已知未做的事（诚实清单）

1. 本引擎是规则层实现，不构成"数字生命已具备个体性"的证据。个体性在我们自己的判据下仍未闭合，这一点在判据核第一条即已声明

2. 核心结论样本量为 n=8 / n=16，未做多重比较校正

3. 尚无外部平台基线对比（Avida / ABM / 细胞自动机）

4. 关键期结论中的"撤除强于全程"机制已定位（不可逆锁死 + 持续投入摊薄），但未做剂量-反应曲线验证

**“所有开源 Agent 框架都在问「它能做什么」。**
**本项目问的是「当它必须取舍、必须记住、而且真的会死的时候，它是什么」。**

这不是一个更好的工具。这是换了一类前提。